

PIDAC Practicumhandleiding

(Datarepresentaties, Tools en Technieken)

PIDAC Practicumregels



Wees voorzichtig met het gebruik van de PIDAC-modules. Deze modules worden niet meer geproduceerd en de losse onderdelen zijn vaak niet meer verkrijgbaar. Reparatie is meestal niet mogelijk. Houd je daarom aan de volgende regels:

- Pak een module direct uit de doos en plaats hem meteen op het experimenteerbord. Laat hem niet rondslingeren.
- Let erop dat je de pinnen aan de achterkant niet buigt of breekt bij het plaatsen.
- Leg modules die je niet meer nodig hebt terug in de doos. **Laat ze niet rondslingeren.**
- Trek stroomdraadjes eruit bij de stekker en niet aan de draad zelf.

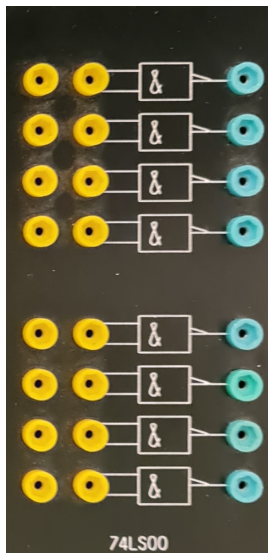
Week 1

Deel 1. Schakelingen en notaties

Je hebt maar een beperkt aantal logische poorten nodig om elke mogelijke schakeling te kunnen bouwen. Het doel van dit deel is om vertrouwd te raken met deze poorten en de manieren waarop we hiermee schakelingen kunnen bouwen. Je leert ook hoe je schakelingen symbolisch kunt beschrijven.

Voor dit practicum krijg je een antwoordformulier waarop je al je antwoorden kunt invullen.

Logische poorten



PIDAC-module met 8 NAND-poorten

De NAND-poort heeft het volgende gedrag: op de uitgang staat 5 volt, tenzij beide ingangen 5 volt ontvangen. Dan staat er 0 volt op de uitgang. Je kunt dit zelf testen met een woordgever en een indicator.



Links: Woordgever. Rechts: Indicator.

Opdracht 1.1

Lees de practicumregels goed door voordat je begint.

Nodig: NAND-module, woordgevermodule, indicatormodule en drie draadjes.

Bouw de schakeling en test of de beschrijving van de NAND-poort klopt.

Eentjes en nulletjes

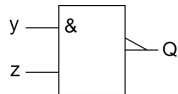
Computers werken met eentjes en nulletjes. Elke poort kan in twee toestanden zijn: 5 volt (hoog, ofwel 1) of 0 volt (laag, ofwel 0).

Niet alle digitale schakelingen werken op 5 volt; sommige gebruiken bijvoorbeeld 3,3 volt als hoog. Maar ongeacht het voltage zijn er altijd twee toestanden.

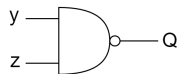
Het gedrag van een logische poort wordt beschreven met een waarheidstabel. Voor een NAND-poort met ingangen y en z en uitgang Q :

y	z	Q
0	0	1
0	1	1
1	0	1
1	1	0

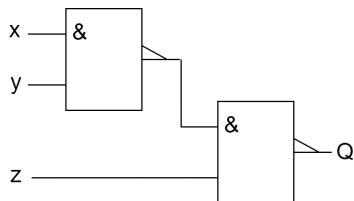
De symbolische notatie van een NAND-poort:



Alternatief symbool:



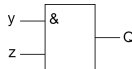
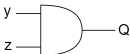
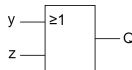
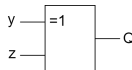
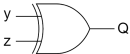
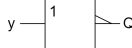
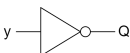
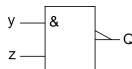
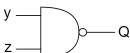
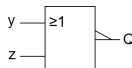
Complexere schakelingen kunnen worden opgebouwd door poorten met elkaar te verbinden:



Opdracht 1.2

Nodig: twee NAND-poorten, een woordgever, een indicator en vijf draadjes.
Bouw de schakeling en vul de waarheidstabel op het antwoordblad in.

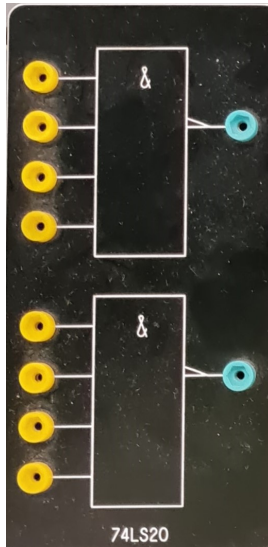
Veelvoorkomende poorten

Poort	Rechthoekige notatie	Alternatieve notatie	Waarheidstabel															
AND			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	y	z	Q	0	0	0	0	1	0	1	0	0	1	1	1
y	z	Q																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	y	z	Q	0	0	0	0	1	1	1	0	1	1	1	1
y	z	Q																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
XOR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	0	0	1	1	1	0	1	1	1	0
y	z	Q																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Inverter			<table><tr><th>y</th><th>Q</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	y	Q	0	1	1	0									
y	Q																	
0	1																	
1	0																	
NAND			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	1	0	1	1	1	0	1	1	1	0
y	z	Q																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR			<table><tr><th>y</th><th>z</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	y	z	Q	0	0	1	0	1	0	1	0	0	1	1	0
y	z	Q																
0	0	1																
0	1	0																
1	0	0																
1	1	0																

Overzicht van veelvoorkomende logische poorten

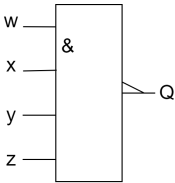
Meerdere ingangen

De bovenstaande poorten hebben allemaal twee ingangen (behalve de inverter), maar in principe kunnen dat er ook meer zijn. Het PIDAC-systeem bevat bijvoorbeeld ook NAND-poorten met vier ingangen.



NAND-module met vier ingangen

De schematische weergave en waarheidstabel van een NAND-poort met vier ingangen ziet er zo uit:

Symbol	Waarheidstabel				
	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>	<i>Q</i>
	0	0	0	0	1
	0	0	0	1	1
	0	0	1	0	1
	0	0	1	1	1
	0	1	0	0	1
	0	1	0	1	1
	0	1	1	0	1
	0	1	1	1	1
	1	0	0	0	1
	1	0	0	1	1
	1	0	1	0	1
	1	0	1	1	1
	1	1	0	0	1
	1	1	0	1	1
	1	1	1	0	1
	1	1	1	1	0

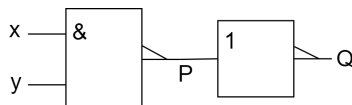
Opdracht 1.3

Nodig: een module met NAND-poorten met vier ingangen, een woordgever-module, een indicatormodule en vijf draadjes.

Experimenteer met de NAND-poort. Ga na of de bovenstaande tabel klopt. Wat gebeurt er als een ingang nergens op is aangesloten?

Poorten samenstellen

In principe is de bovenstaande verzameling poorten redundant. Dat wil zeggen, je kunt alle poorten maken met een combinatie van andere poorten. Bijvoorbeeld, het schema hieronder heeft hetzelfde gedrag als een AND-poort:



We kunnen dit zien aan de hand van de waarheidstabel:

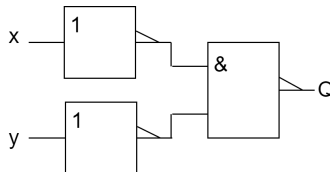
x	y	P	Q
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Opdracht 1.4

Je kunt een NAND-poort als inverter gebruiken. Maak een schakeling die dat doet. Gebruik een woordgever en een indicator om je schakeling te testen. Teken het schema op het antwoordformulier en maak een waarheidstabel om te verifiëren dat het klopt.

Opdracht 1.5

Bouw het onderstaande schema na:



Maak een waarheidstabel op je antwoordformulier. Met welke logische poort komt deze schakeling overeen?

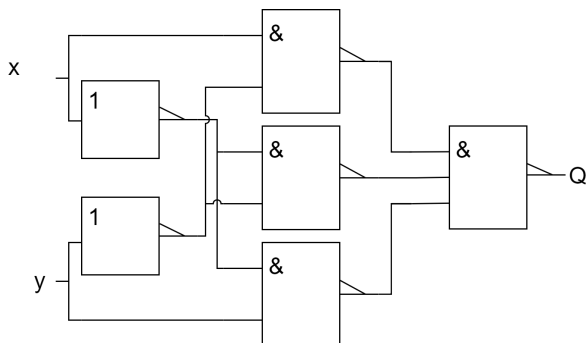
Opdracht 1.6

Maak een schakeling die het gedrag van een NAND-poort met drie ingangen nabootst. Gebruik twee NAND-poorten met twee ingangen en een inverter. Gebruik een woordgever en een indicator om je schakeling te testen. Teken het schema op het antwoordformulier en maak een waarheidstabel om te verifiëren dat het klopt.

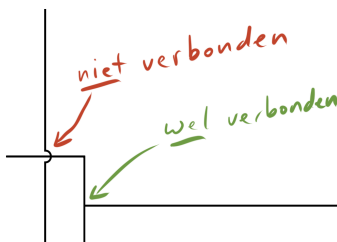
Tip: je kunt een inverter en een NAND-poort gebruiken om een AND-poort te maken. Vervolgens kun je een AND-poort en een NAND-poort combineren tot een NAND-poort met drie ingangen.

Opdracht 1.7

Bouw het onderstaande schema na:



De verbindingen van de onderste inverter naar de bovenste twee NAND-poorten bevatten boogjes. Deze geven aan dat de betreffende verbinding een andere kruist zonder ermee verbonden te zijn:



Maak een waarheidstabel op je antwoordformulier. Vereenvoudig daarna de schakeling. Dat wil zeggen: maak een schakeling die minder poorten gebruikt, maar hetzelfde doet. Teken het schema op je antwoordformulier.

Voor deze schakeling heb je een NAND-poort met drie ingangen nodig. Er is geen PIDAC-module die zo'n poort heeft, maar je kunt hiervoor een NAND-poort met vier ingangen gebruiken.

Aftekenen! Laat opdrachten 1.1 - 1.7 aftekenen.

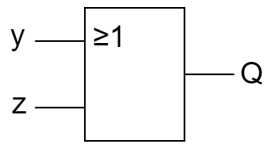
Boole-notatie

Het bouwen van simpele schakelingen kan vaak op basis van intuïtie en uitproberen. Maar bij complexere schakelingen werkt dat niet meer. Daarvoor gebruiken we Boole-algebra. Boole-algebra is in 1854 ontwikkeld door George Boole.

Als je ervaring hebt met propositielogica, zal Boole-algebra je bekend voorkomen. Het is namelijk equivalent — alleen de notatie verschilt.

We duiken niet volledig in de algebra, maar we leren hoe je Boole-notatie kunt gebruiken om schakelschema's te beschrijven.

Voor een OR-poort gebruiken we bijvoorbeeld het symbool $+$. Dus het volgende schema:



Schrijven we als: $Q = y + z$

Hieronder een overzicht van de notatie:

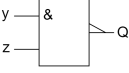
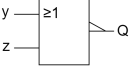
Poort	Boole-notatie	Schema
OR	$Q = y + z$	
AND	$Q = y \cdot z$	
Inverter	$Q = \overline{y}$	
XOR	$Q = y \oplus z$	

Boole-notatie voor logische poorten

Let op: je hebt het symbool $\oplus$ strikt genomen niet nodig; je kunt XOR ook herschrijven met behulp van $\cdot$, $+$ en $\overline{}$.

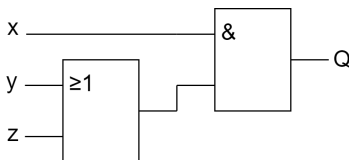
Gebruik Boole-notatie

We kunnen ook NAND- en NOR-poorten beschrijven:

Poort	Boole-notatie	Schema
NAND	$Q = \overline{y \cdot z}$	
NOR	$Q = \overline{y + z}$	

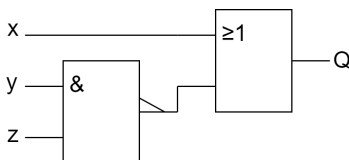
Complexe schakelingen

Een schakeling als deze:



Schrijven we als: $Q = x \cdot (y + z)$

Een andere schakeling:



Wordt: $Q = x + \overline{y \cdot z}$

Operatorvolgorde: $\bar{a}$ gaat voor $a \cdot b$, dat gaat weer voor $a + b$. Bij gelijke operatoren wordt van links naar rechts geëvalueerd.

Voorbeeld: $a \cdot (b + c) \cdot e + f$

Opdracht 1.8

Bekijk de expressie: $Q = \overline{y \cdot z}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Opdracht 1.9

Bekijk de expressie: $Q = \overline{(a \oplus b) \cdot c}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Opdracht 1.10

Bekijk de expressie: $Q = \overline{\overline{p \cdot q} \cdot \overline{p \cdot q}}$

Teken het bijbehorende schema, bouw het na en maak een waarheidstabel.

Aftekenen! Laat opdrachten 1.8 - 1.10 aftekenen.

Boole-expressies evalueren

Je kunt aan de hand van Boole-expressies direct een waarheidstabel opstellen, zonder de schakeling eerst te bouwen. Voor eenvoudige expressies zoals $Q = y + z$, $Q = y \cdot z$ en $Q = \overline{y}$ kun je de waarheidstabellen van de poorten raadplegen (zie de cheatsheet).

Voor complexere expressies vul je gewoon 1 of 0 in voor de variabelen en kijk je wat er uitkomt. Neem bijvoorbeeld $Q = \overline{y + z}$. Dit is de expressie van een NOR-poort.

Wat gebeurt er als $y = 1$ en $z = 0$?

$$Q = \overline{y + z} = \overline{1 + 0} = \overline{1} = 0$$

Doe dit voor alle combinaties van y en z :

y	z	Q
0	0	1
0	1	0
1	0	0
1	1	0

Opdracht 1.11

We hebben een schakeling gegeven door:

$$Q = \overline{v \cdot \overline{w}} \oplus p$$

Beredeneer de waarheidstabel. Als je klaar bent, teken het schema, bouw het en controleer of je waarheidstabel klopt.

Opdracht 1.12

De schakeling is gegeven door:

$$Q = \overline{\overline{a} + b} + c$$

Beredeneer de waarheidstabel voor deze schakeling.

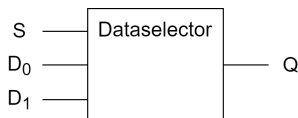
Opdracht 1.13

NAND-poorten hebben de handige eigenschap dat je er elke andere poort mee kunt bouwen. Ontwerp een schakeling van alleen NAND-poorten die zich gedraagt als een AND-poort. Teken het schema in je antwoordformulier.

Tip: je kunt een NAND als inverter gebruiken door slechts één ingang te verbinden.

Opdracht 1.14

Een veelgebruikte schakeling is de dataselector (multiplexer). Stel je een systeem voor met twee datasignalen en een digitale selectielijn. Afhankelijk van de waarde van de selectielijn gaat het ene of het andere signaal door.



Inputs: D_0 , D_1 , S (select). Output: Q

S	D_0	D_1	Q
0	d_0	d_1	d_0
1	d_0	d_1	d_1

Volledig uitgewerkt:

S	D_0	D_1	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Ontwerp een schakeling voor deze tabel.

Aftekenen! Laat opdrachten 1.11 - 1.14 aftekenen.

Opmerkingen

Relatie met programmeertalen

De Boole-logica lijkt op wat je tegenkomt in programmeertalen:

- In plaats van 1 en 0 zie je vaak `true` en `false`
- $a + b$ wordt `a || b` in C, of `a or b` in Python
- $a \cdot b$ wordt `a && b` in C, of `a and b` in Python
- $\bar{a}$ wordt `!a` in C, of `not a` in Python

Relatie met propositielogica

Boole-algebra is gelijkwaardig aan propositielogica:

- 1 en 0 komen overeen met $\top$ (waar) en $\perp$ (onwaar)
- $a + b$ wordt $a \vee b$
- $a \cdot b$ wordt $a \wedge b$
- $\bar{a}$ wordt $\neg a$

Week 2

Boolealgebra: de kracht van abstractie

In deze sessie leer je hoe je Boole-algebra kunt gebruiken om schakelingen te ontwerpen. In de vorige sessie heb je geleerd wat het verband is tussen een Boole-expressie en een schakeling. De kracht van Boole-algebra is dat je ermee kunt rekenen: je kunt het gebruiken om systematisch schakelingen te ontwerpen en te vereenvoudigen.

In het eerste deel van deze sessie introduceren we de rekenregels van de Boole-algebra. Daarna gebruiken we deze regels om schakelingen te ontwerpen.

Herhaling

Opdracht 2.1

We hebben een schakeling gegeven door $Q = (a + \bar{a}) \cdot b$.

Beredeneer de waarheidstabel voor deze schakeling. Teken het schema op je antwoordblad, bouw de schakeling en test of je waarheidstabel klopt.

Schakelingen vereenvoudigen

De bovenstaande schakeling is onnodig ingewikkeld. We kunnen het eenvoudiger maken. Als je naar de waarheidstabel kijkt, zie je wellicht dat deze gelijk is aan $Q = b$. Dit kunnen we ook beredeneren:

Beschouw het deel $a + \bar{a}$:

a	$a + \bar{a}$
0	1
1	1

Met andere woorden: $a + \bar{a} = 1$. Dus kunnen we de expressie $Q = (a + \bar{a}) \cdot b$ vereenvoudigen naar $Q = 1 \cdot b$. En dat kunnen we weer verder vereenvoudigen:

b	$1 \cdot b$
0	0
1	1

Hieruit volgt: $1 \cdot b = b$. We hebben dus geen logische poorten nodig; we kunnen de drukknop van de woordgever direct met de indicator verbinden.

Samengevat:

$$Q = (a + \bar{a}) \cdot b = 1 \cdot b = b$$

We hebben hiermee twee fundamentele regels ontdekt:

$$\begin{aligned} z + \bar{z} &= 1 \\ 1 \cdot z &= z \end{aligned}$$

We kunnen heel veel meer van dit soort wetmatigheden vaststellen door schakelingen te bouwen en daar waarheidstabellen mee te maken. Hieronder volgen een aantal van deze wetmatigheden:

Rekenregels voor variabelen

$$\begin{array}{ll} 1. & z + z = z \\ 2. & z \cdot z = z \\ 3. & \bar{\bar{z}} = z \end{array} \quad \begin{array}{ll} 4. & z + \bar{z} = 1 \\ 5. & z \cdot \bar{z} = 0 \end{array}$$

Rekenregels voor constanten

$$\begin{array}{lll} 6. & 0 + 0 = 0 & 10. & 0 \cdot 0 = 0 & 14. & \bar{0} = 1 \\ 7. & 0 + 1 = 1 & 11. & 0 \cdot 1 = 0 & 15. & \bar{1} = 0 \\ 8. & 1 + 0 = 1 & 12. & 1 \cdot 0 = 0 \\ 9. & 1 + 1 = 1 & 13. & 1 \cdot 1 = 1 \end{array}$$

Moduluswetten

$$\begin{array}{ll} 16. & 0 + z = z \\ 17. & 1 + z = 1 \end{array} \quad \begin{array}{ll} 18. & 0 \cdot z = 0 \\ 19. & 1 \cdot z = z \end{array}$$

Opdracht 2.2

Verifieer aan de hand van een waarheidstabel dat de rekenregel $z \cdot \bar{z} = 0$ klopt.

Je kan de bovenstaande regels gebruiken om, bijvoorbeeld de expressie $Q = p + \bar{0} \cdot \bar{p}$ vereenvoudigen.

$$\begin{aligned} Q &= p + \bar{0} \cdot \bar{p} \\ &= p + \bar{1} \cdot \bar{p} && \text{(regel 14)} \\ &= p + \bar{\bar{p}} && \text{(regel 19)} \\ &= p + p && \text{(regel 3)} \\ &= p && \text{(regel 1)} \end{aligned}$$

Opdracht 2.3

Vereenvoudig de expressie $Q = \overline{a \cdot \overline{a}}$. Gebruik de bovenstaande wetten en schrijf alle stappen van je afleiding op. Vergelijk beide expressies met behulp van een waarheidstabel.

Opdracht 2.4

Vereenvoudig de expressie $Q = a \cdot \overline{a \cdot \overline{\overline{a}}}$. Geef een volledige afleiding met verwijzing naar de gebruikte regels.

Wetmatigheden uit de wiskunde

Er zijn een aantal wetmatigheden die overeenkomen met wat je in de wiskunde gewend bent.

Associativiteit

Voor een herhaling van dezelfde operator maakt de volgorde waarin je ze toepast niet uit:

$$20. \quad (x + y) + z = x + (y + z)$$

$$21. \quad (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

Commutativiteit

Je kunt de volgorde van de argumenten van de operatoren omdraaien:

$$22. \quad y + z = z + y$$

$$23. \quad y \cdot z = z \cdot y$$

Distributieve wetten

Voor $\cdot$ en $+$ zijn de volgende distributieregels van toepassing:

$$24. \quad x \cdot (y + z) = x \cdot y + x \cdot z$$

$$25. \quad x + y \cdot z = (x + y) \cdot (x + z)$$

Opdracht 2.5

Laat met waarheidstabellen zien dat distributieve wet 25 correct is.

Opdracht 2.6

De expressie $Q = c + (a \cdot b + 0) \cdot \bar{a}$ is equivalent aan $Q = c$.

Geef de afleiding die laat zien dat dit correct is. Schrijf alle stappen van je afleiding op en geef aan welke regels je gebruikt. Gebruik alleen de volgende wetmatigheden:

Regel 5: $z \cdot \bar{z} = 0$

Regel 16: $0 + z = z$

Regel 18: $0 \cdot z = 0$

Regel 21: $(x \cdot y) \cdot z = x \cdot (y \cdot z)$

Regel 22: $y + z = z + y$

Regel 23: $y \cdot z = z \cdot y$

Aftekenen! Laat opdracht 2.1 - 2.6 aftekenen.

Absorptie en De Morgan

Nog een laatste serie van wetmatigheden.

Absorptiewetten

De onderstaande wetten geven handige shortcuts bij het vereenvoudigen van complexe Boole-expressies:

$$26. \quad z + y \cdot z = z$$

$$27. \quad z \cdot (y + z) = z$$

$$28. \quad y + \overline{y} \cdot z = y + z$$

$$29. \quad y \cdot (\overline{y} + z) = y \cdot z$$

De Morganwetten

De wetten van De Morgan beschrijven de relatie tussen de drie operatoren:

$$30. \quad \overline{y + z} = \overline{y} \cdot \overline{z}$$

$$31. \quad \overline{\overline{y} \cdot \overline{z}} = \overline{\overline{y}} + \overline{\overline{z}}$$

$$32. \quad y + z = \overline{\overline{y} \cdot \overline{z}}$$

$$33. \quad y \cdot z = \overline{\overline{y} + \overline{z}}$$

Opdracht 2.7

Laat met waarheidstabellen zien dat regel 30 correct is.

De hoeveelheid wetten lijkt misschien wat veel, maar de wetten zijn zeer redundant. Dat wel zeggen dat je de meeste wetten uit de andere wetten kan afleiden. Bijvoorbeeld, regel 28 ($y + \overline{y} \cdot z = y + z$) kan je als volgt afleiden:

$$\begin{aligned} y + \overline{y} \cdot z &= (y + \overline{y}) \cdot (y + z) && \text{(regel 25)} \\ &= 1 \cdot (y + z) && \text{(regel 4)} \\ &= y + z && \text{(regel 19)} \end{aligned}$$

Opdracht 2.8

De regels 32 en 33 (De Morgan) zijn redundant. Laat zien dat je regel 32 kunt afleiden met behulp van regel 30 en regel 3.

Waarheidstabellen en expressies ontwerpen

Dat was een vrij theoretisch uitstapje, maar we gaan nu eindelijk zien hoe we deze inzichten kunnen gebruiken voor het ontwerpen van schakelingen. De truc

die je gaat leren is hoe je Boole-algebra kunt gebruiken om op een systematische manier waarheidstabellen om te zetten in schakelingen.

Neem de volgende expressie met twee variabelen:

$$Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot \bar{z} + y \cdot z$$

Deze uitdrukking is altijd waar. Ze bestaat uit vier losse disjuncten (delen gescheiden door +): $\bar{y} \cdot \bar{z}$, $\bar{y} \cdot z$, $y \cdot \bar{z}$ en $y \cdot z$. Voor elke combinatie van y en z is precies één disjunct gelijk aan 1 en de rest aan 0. Bijvoorbeeld: als $y = 0$ en $z = 1$, dan is $\bar{y} \cdot z = 1$, en de rest 0. Omdat het een OR-expressie is, is $Q = 1$.

Stel dat we willen dat Q altijd de waarde 1 heeft, behalve als $y = 1$ en $z = 0$. We schrappen dan gewoon de bijbehorende disjunct:

$$Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot z$$

Opdracht 2.9 Maak een waarheidstabel en ga na dat de bovenstaande expressie altijd waar is, behalve als $y = 1$ en $z = 0$.

We kunnen dus voor elke invulling van y en z een expressie formuleren die alleen waar is voor die specifieke combinatie. Hiermee kunnen we expressies opstellen op basis van waarheidstabellen.

Stel dat we de volgende waarheidstabel willen implementeren:

y	z	Q
0	0	1
0	1	1
1	0	1
1	1	0

De rijen waarvoor $Q = 1$ zijn:

- $\bar{y} \cdot \bar{z}$
- $\bar{y} \cdot z$
- $y \cdot \bar{z}$

Samen geeft dit:

$$Q = \bar{y} \cdot \bar{z} + \bar{y} \cdot z + y \cdot \bar{z}$$

We kunnen dit vereenvoudigen:

$$\begin{aligned}
 Q &= \overline{y} \cdot (\overline{z} + z) + \overline{z} \cdot y \\
 &= \overline{y} \cdot 1 + \overline{z} \cdot 1 \\
 &= \overline{y} + \overline{z} \\
 &= \overline{y \cdot z} \quad (\text{regel 31})
 \end{aligned}$$

Met andere woorden: deze waarheidstabel komt overeen met een enkele NAND-poort.

Voor schakelingen met meer dan twee inputs wordt het moeilijker om dit intuïtief te doen. Neem bijvoorbeeld de dataschakelaar (multiplexer):

D_0	D_1	S	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

De disjuncten met $Q = 1$ zijn:

- $\overline{D_0} \cdot D_1 \cdot S$
- $D_0 \cdot \overline{D_1} \cdot \overline{S}$
- $D_0 \cdot D_1 \cdot \overline{S}$
- $D_0 \cdot D_1 \cdot S$

Samen vormen ze:

$$Q = \overline{D_0} \cdot D_1 \cdot S + D_0 \cdot \overline{D_1} \cdot \overline{S} + D_0 \cdot D_1 \cdot \overline{S} + D_0 \cdot D_1 \cdot S$$

Vereenvoudiging:

$$\begin{aligned}
 Q &= S \cdot D_1 \cdot \overline{D_0} + S \cdot D_1 \cdot D_0 + \overline{S} \cdot D_0 \cdot \overline{D_1} + \overline{S} \cdot D_0 \cdot D_1 \\
 &= S \cdot (D_1 \cdot \overline{D_0} + D_1 \cdot D_0) + \overline{S} \cdot (D_0 \cdot \overline{D_1} + D_0 \cdot D_1) \\
 &= S \cdot (D_1 \cdot (\overline{D_0} + D_0)) + \overline{S} \cdot (D_0 \cdot (\overline{D_1} + D_1)) \\
 &= S \cdot D_1 + \overline{S} \cdot D_0
 \end{aligned}$$

Opdracht 2.10 Schrijf de expressie van de dataschakelaar om zodat je een uitdrukking overhoudt die je met alleen NAND-poorten kunt maken. Laat de afleiding op het antwoordblad zien.

Bouw de dataschakelaar en teken het schema op je antwoordblad.

Aftekenen! Laat opdracht 2.7 - 2.10 aftekenen.

Opdracht 2.11 Bekijk de volgende waarheidstabel:

p	q	r	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Maak een schakeling voor deze waarheidstabel. De eenvoudigste oplossing gebruikt twee inverters en één OR-poort. Omdat PIDAC geen OR-poorten heeft, moet je de expressie herschrijven met drie inverters en twee NAND-poorten. Geef de afleiding en het schema op je antwoordblad.

Meerdere outputs

Tot nu toe hebben we alleen schakelingen met één output gezien. Wat als we een schakeling willen maken met meerdere outputs? De makkelijkste manier is om voor elke output een aparte schakeling te ontwerpen.

Neem bijvoorbeeld een schakeling die twee 1-bits binaire getallen optelt: een *half adder*. De mogelijke berekeningen zijn:

a	b	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Opdracht 2.12 Ontwerp een schakeling voor de *half adder*.

- Maak twee waarheidstabellen: één voor de linkerbit (carry), één voor de rechterbit (sum).
- Stel Boole-expressies op voor beide outputs.
- Vereenvoudig indien nodig de expressies. (Houd er rekening mee dat je alleen XOR-poorten en NAND-poorten te beschikking hebt.)
- Bouw de schakeling.

Aftekenen! Laat opdracht 2.11 en 2.12 aftekenen.

Week 3

Rekenen met logica

Half Adder

Vorige week ben je geëindigd met een *half adder*, een schakeling waarmee je twee 1-bits getallen kunt optellen.

De vier mogelijke berekeningen met deze rekenmachine zijn:

$$0 + 0 = 00, \quad 0 + 1 = 01, \quad 1 + 0 = 01, \quad 1 + 1 = 10$$

De gecombineerde waarheidstabel voor de linkerbit (LB) en rechterbit (RB) is:

i_1	i_2	LB	RB
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Voor LB:

i_1	i_2	LB
0	0	0
0	1	0
1	0	0
1	1	1

Voor RB:

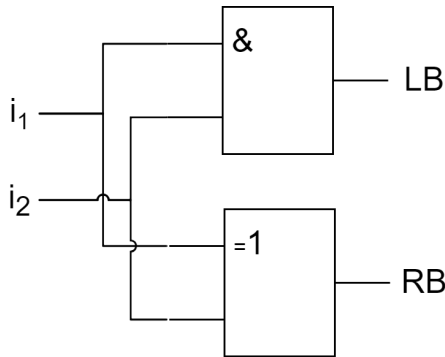
i_1	i_2	RB
0	0	0
0	1	1
1	0	1
1	1	0

Dit geeft de volgende expressies:

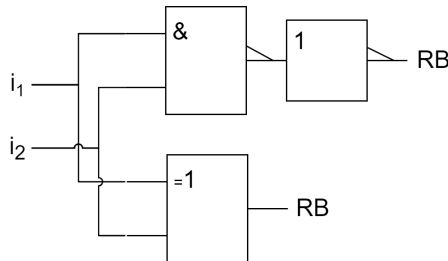
$$\begin{aligned} LB &= i_1 \cdot i_2 \\ RB &= \overline{i_1} \cdot i_2 + i_1 \cdot \overline{i_2} \end{aligned}$$

De expressie voor RB kan ook worden geschreven als:

$$RB = i_1 \oplus i_2$$



Aangezien PIDAC geen AND-poort heeft, lossen we dit anders op:



Full Addder

Een *full addder* kan drie 1-bits getallen optellen. De output is een 2-bits getal.

Opdracht 3.1

Vul de waarheidstabel voor de *full addder* op het antwoordblad in.

Opdracht 3.2

Ga na met de waarheidstabel dat:

$$RB = i_1 \cdot \overline{i_2} \cdot \overline{i_3} + \overline{i_1} \cdot i_2 \cdot \overline{i_3} + \overline{i_1} \cdot \overline{i_2} \cdot i_3 + i_1 \cdot i_2 \cdot i_3$$

Opdracht 3.3

Bewijs met de waarheidstabel dat:

$$RB = (i_1 \oplus i_2) \oplus i_3$$

Opdracht 3.4

Geef aan de hand van de waarheidstabel de (niet vereenvoudigde) expressie voor LB .

Opdracht 3.5

De expressie voor de linkerbit kan worden herschreven als:

$$LB = \overline{i_1 \cdot i_2} \cdot \overline{(i_1 \oplus i_2) \cdot i_3}$$

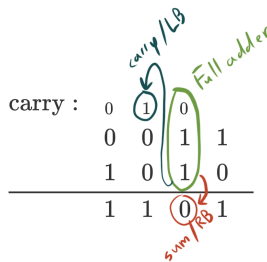
Bouw een *full adder* met twee XOR- en drie NAND-poorten. Label de poorten met i_1 , i_2 , i_3 , LB (Carry) en RB (Sum).

Teken het schema op je antwoordblad.

Aftekenen! Laat opdracht 3.1 - 3.5 aftekenen.

Optelschakeling voor grotere getallen

Voor optelling van grotere getallen combineren we half en full adders.



Opdracht 3.6

Vind twee of drie andere groepjes en maak gezamenlijk een 4-bits optelschakeling met *full adders*.

Zorg voor labeling van inputs, outputs en volgorde van de adders.

Teken het schema op je antwoordblad.

Let op: er kan overflow ontstaan. Zorg voor een vijfde output die overflow aangeeft.

Aftekenen! Laat opdracht 3.6 aftekenen.

Optel-/aftrekschakeling

We bouwen nu een schakeling die zowel kan optellen als aftrekken met behulp van two's complement.

Bij two's complement wordt aftrekken gerealiseerd door bitflipping en het optellen van 1.

De flip-waarheidstabel per bit:

$SUB/\overline{ADD}$	Input bit	Output bit
0	0	0
0	1	1
1	0	1
1	1	0

Opdracht 3.7

Met welke logische poort kun je deze waarheidstabel realiseren?

Opdracht 3.8

Maak een 4-bits optel-/aftrekschakeling. De schakeling heeft 8 inputs en een $SUB/\overline{ADD}$ -selectiebit.

Als $SUB/\overline{ADD} = 0$ wordt er opgeteld, als $SUB/\overline{ADD} = 1$ wordt er afgetrokken.

Teken het schema op je antwoordblad.

Let op: overflow mag worden genegeerd.

Aftekenen! Laat opdracht 3.7 en 3.8 aftekenen.